

EDIT_FORMATS_USER_GUIDE

Edit Formats User Guide

HelixCode LLM Response Formats

Version: 1.0 **Last Updated:** November 7, 2025 **Feature Status:** Production Ready

Table of Contents

1. [Introduction](#)
 2. [Quick Start](#)
 3. [Format Overview](#)
 4. [Whole-File Format](#)
 5. [Diff Format](#)
 6. [Git Unified Diff \(UDiff\) Format](#)
 7. [Search/Replace Format](#)
 8. [Editor Format](#)
 9. [Architect Format](#)
 10. [Ask Format](#)
 11. [Line Number Format](#)
 12. [Format Auto-Detection](#)
 13. [Best Practices](#)
 14. [Troubleshooting](#)
 15. [FAQ](#)
-

Introduction

Edit Formats define how LLMs (Language Learning Models) communicate file changes to HelixCode. Different formats suit different use cases - from simple whole-file replacements to complex architectural changes.

Why Multiple Formats?

- **Flexibility:** Choose the right format for your task
- **Efficiency:** Small changes don't need whole-file replacements
- **Clarity:** Structured formats make changes easy to review
- **Safety:** Ask mode allows confirmation before changes

- **Compatibility:** Works with all 13 supported LLM providers

Supported Formats

Format	Best For	Complexity
Whole-File	Small files, new files	Simple
Diff	Precise changes with context	Medium
UDiff	Git workflows, version control	Medium
Search/Replace	Find and replace operations	Simple
Editor	Line-specific edits	Medium
Architect	High-level refactoring	Simple
Ask	Clarification, confirmation	Simple
Line Number	Direct line editing	Medium

Quick Start

Using a Format

```
# Tell HelixCode which format to use
helix --format whole "Create a Hello World program"

# Or let auto-detection figure it out
helix "Update the timeout in config.yaml to 60 seconds"
```

Format in API

```
import "dev.helix.code/internal/editor/formats"

// Create format registry
registry, _ := formats.RegisterAllFormats()

// Parse LLM response
edits, format, _ := registry.ParseWithAutoDetect(ctx,
    llmResponse)

// Apply edits
for _, edit := range edits {
    applyEdit(edit)
}
```

Format Overview

Format Selection Guide

Choose Whole-File when: - Creating new files - File is small (<100 lines) - Complete rewrite needed

Choose Diff/UDiff when: - Making precise changes - Need to see context - Working with version control

Choose Search/Replace when: - Simple find/replace operations - Updating configuration values - Renaming variables

Choose Editor when: - Editing specific lines - Multiple small changes - Line-based operations

Choose Architect when: - Restructuring project - Creating/deleting files - High-level refactoring

Choose Ask when: - Need clarification - Unsure about approach - Want user confirmation

Choose Line Number when: - Direct line editing - Reviewing numbered output - Precise line-by-line control

Whole-File Format

Replace entire file content. Simplest format, best for small files.

Syntax

```
File: <file_path>
```go
<complete file content>
```

### Examples

**\*\*Creating a new file:\*\***

File: src/hello.go

```
package main
```

```
import "fmt"
```

```
func main() {
 fmt.Println("Hello, World!")
}
```

**\*\*Updating a small file:\*\***

File: config.yaml

```
server:
 port: 8080
 timeout: 60

database:
 host: localhost
 port: 5432
```

### ### Advantages

- Simple and clear
- Easy to review
- No ambiguity

### ### Disadvantages

- Inefficient for large files
- Large token usage
- Can't see what changed

### ### When to Use

- Files under 100 lines
- Creating new files
- Complete rewrites
- Large files with small changes
- Need to show context

---

## ## Diff Format

Standard unified diff format with context lines.

### ### Syntax

— a/ +++ b/ @@ -old\_start,old\_count +new\_start,new\_count @@ context line  
-removed line +added line context line

### ### Examples

**\*\*Adding an import:\*\***

— a/src/main.go +++ b/src/main.go @@ -1,5 +1,6 @@ package main

import "fmt" +import "os"

func main() {

**\*\*Updating a function:\*\***

— a/src/utils.go +++ b/src/utils.go @@ -10,7 +10,8 @@ func calculateTotal(items  
[]Item) float64 { total := 0.0 for \_, item := range items { - total += item.Price + //  
Apply tax + total += item.Price \* 1.1 } return total }

### ### Advantages

- Shows context
- Clear what changed
- Standard format
- Good for review

### ### Disadvantages

- More verbose than search/replace
- Need to know line numbers
- Can be complex for large changes

### ### When to Use

Precise changes with context  
Code reviews  
Multiple related changes  
Simple find/replace  
Whole-file rewrites

---

## ## Git Unified Diff (UDiff) Format

Git-style unified diff with extended headers.

### ### Syntax

```
diff -git a/ b/ index .. — a/ +++ b/ @@ -old_start,old_count +new_start,new_count
@@ context -removed +added
```

### ### Examples

**\*\*Creating a new file:\*\***

```
diff -git a/src/models/user.go b/src/models/user.go new file mode 100644 index
0000000..a1b2c3d — /dev/null +++ b/src/models/user.go @@ -0,0 +1,8 @@ +package
models + +type User struct { + ID int + Username string + Email string +}
```

**\*\*Renaming a file:\*\***

```
diff -git a/old_name.go b/new_name.go similarity index 100% rename from
old_name.go rename to new_name.go
```

**\*\*Deleting a file:\*\***

```
diff -git a/deprecated.go b/deprecated.go deleted file mode 100644 index
a1b2c3d..0000000 — a/deprecated.go +++ /dev/null
```

### ### Advantages

- Full git metadata
- Supports file operations
- Version control friendly
- Industry standard

### ### Disadvantages

- More verbose
- Complex syntax
- Requires git knowledge

### ### When to Use

- Git workflows
- File operations (create/delete/rename)
- Version control integration
- Patch generation
- Quick edits
- Non-git projects

---

## ## Search/Replace Format

Regex-based find and replace operations.

### ### Syntax

**Block Style:**

File: <<<<<< SEARCH ===== >>>>>> REPLACE

**Keyword Style:**

File: SEARCH: REPLACE:

**Inline Style:**

File: search: replace:

### ### Examples

**Simple replacement:**

File: config.yaml <<<<<< SEARCH timeout: 30 ===== timeout: 60 >>>>>>  
REPLACE

**Function replacement:**

File: src/auth.go <<<<<< SEARCH func authenticate(user string) bool { return  
false } ===== func authenticate(user string, token string) bool { return  
validateToken(user, token) } >>>>>> REPLACE

**\*\*Multiple replacements:\*\***

File: README.md SEARCH: version 1.0 REPLACE: version 2.0

File: package.json search: "version": "1.0.0" replace: "version": "2.0.0"

### ### Advantages

- Simple and intuitive
- Exact matching
- Works for any text
- Multiple replacements

### ### Disadvantages

- Must match exactly (whitespace matters)
- No context shown
- Can match wrong occurrence

### ### When to Use

- Simple text replacement
- Configuration updates
- Variable renaming
- Known exact text
- Need to see context
- Complex structural changes

---

## ## Editor Format

Line-based editing with insert, delete, and replace operations.

### ### Syntax

File: INSERT AT LINE :

DELETE LINE DELETE LINE -

REPLACE LINE :

**\*\*Alternative compact format:\*\***

L:

### ### Examples

#### **\*\*Inserting lines:\*\***

File: src/main.go INSERT AT LINE 5: import "os" import "log"

INSERT AT LINE 20: // TODO: Add error handling

#### **\*\*Deleting lines:\*\***

File: config.yaml DELETE LINE 10

DELETE LINE 20-25

#### **\*\*Replacing lines:\*\***

File: src/utls.go REPLACE LINE 15: func calculateTotal(items []Item, tax float64) float64 {

REPLACE LINE 30: return total \* (1 + tax)

#### **\*\*Compact format:\*\***

File: test.txt L1: First line updated L5: Fifth line updated L10: L15: Fifteenth line updated

### ### Advantages

- Precise line control
- Multiple operations
- Clear intent
- Compact notation available

### ### Disadvantages

- Need to know line numbers
- No context shown
- Order matters

### ### When to Use

```
Known line numbers
Multiple small edits
Line-specific changes
Scripted edits
Large changes
Don't know line numbers
```

---

## ## Architect Format

High-level structural changes and file operations.

### ### Syntax

CREATE FILE:

<file content>

MODIFY FILE: Changes: -

DELETE FILE:

RENAME FILE: TO

MOVE FILE: TO

### ### Examples

**\*\*Creating files:\*\***

CREATE FILE: src/models/user.go

**package** models

```
type User struct {
 ID int
 Username string
 Email string
}
```

```
func NewUser(username, email string) *User {
 return &User{
 Username: username,
 Email: email,
```

```
}
}
```

**\*\*Describing modifications:\*\***

MODIFY FILE: src/main.go Changes: - Add user authentication middleware -  
Update route handlers to use new User model - Add error handling for database  
operations - Implement graceful shutdown

**\*\*File operations:\*\***

DELETE FILE: src/legacy/old\_module.go

RENAME FILE: config.yml TO config.yaml

MOVE FILE: utils/helper.go TO src/utils/helper.go

**\*\*Multiple operations:\*\***

CREATE FILE: src/middleware/auth.go

**package** middleware

```
func AuthMiddleware() gin.HandlerFunc {
 return func(c *gin.Context) {
 // Authentication logic
 c.Next()
 }
}
```

MODIFY FILE: src/main.go Changes: - Import new auth middleware - Add  
middleware to router - Update error handling

DELETE FILE: src/old\_auth.go

RENAME FILE: config.json TO config.yaml

**### Advantages**

- High-level view
- Describes intent
- Multiple operations
- Clear structure

### ### Disadvantages

- Less precise
- Descriptions need implementation
- No exact code shown

### ### When to Use

- Project restructuring
- File operations
- High-level planning
- Architecture changes
- Precise code edits
- Small changes

---

## ## Ask Format

Question and confirmation mode before making changes.

### ### Syntax

QUESTION: ? File: Context:

PROPOSED CHANGE: File: Description: Rationale:

CLARIFICATION: ?

CONFIRM: for ?

### ### Examples

**Asking for clarification:**

QUESTION: Should I use a mutex or a channel for concurrency control? File: src/worker/pool.go Context: Managing access to the shared worker queue between goroutines

**Proposing changes:**

PROPOSED CHANGE: File: src/auth/middleware.go Description: Add JWT token validation middleware with RSA signature verification Rationale: Current implementation doesn't validate token signatures, which is a security risk

PROPOSED CHANGE: File: src/config/config.go Description: Add support for environment-specific configuration files Rationale: Need different settings for dev, staging, and production

**\*\*Requesting confirmation:\*\***

CONFIRM: Delete the deprecated helper functions for utils/legacy.go?

CONFIRM: Refactor the authentication system to use OAuth2 for src/auth/?

**\*\*Multiple questions:\*\***

QUESTION: Which error handling strategy do you prefer? File: src/handlers/api.go  
Context: Currently using panic for errors

Options: - Return errors (idiomatic Go) - Use panic (current approach) - Custom error types

CLARIFICATION: Should I maintain backward compatibility with the old API?

PROPOSED CHANGE: File: src/api/v2/handlers.go Description: Create new v2 API with improved error handling Rationale: Clean slate without breaking existing clients

### ### Advantages

- Prevents mistakes
- Clarifies ambiguity
- Gets user input
- Safe approach

### ### Disadvantages

- Requires interaction
- Slower workflow
- Not automated

### ### When to Use

Unsure about approach  
Ambiguous requirements  
Major architectural decisions  
Risk of breaking changes  
Simple, clear tasks

## Automated workflows

---

### ## Line Number Format

Direct line editing with numbered line prefixes.

### ### Syntax

File: 1| 2| 3| ...

**\*\*Alternative separators:\*\***

- ``<num>: <content>``
- ``<num> <content>``

### ### Examples

**\*\*Complete file with line numbers:\*\***

File: src/config.go 1| package config 2| 3| import ( 4| "fmt" 5| "os" 6| ) 7| 8| type  
Config struct { 9| Host string 10| Port int 11| } 12| 13| func Load() (\*Config, error)  
{ 14| return &Config{ 15| Host: "localhost", 16| Port: 8080, 17| }, nil 18| }

**\*\*Using colon separator:\*\***

File: config.yaml 1: server: 2: host: localhost 3: port: 8080 4: timeout: 60 5: 6:  
database: 7: host: localhost 8: port: 5432 9: name: mydb

### ### Advantages

- Complete control
- See all lines
- Easy to review
- Clear structure

### ### Disadvantages

- Verbose for large files
- Must include all lines
- High token usage

### ### When to Use

- Small files
- Need full view
- Line-by-line review
- Complete rewrites
- Large files
- Small changes

— — —

## ## Format Auto-Detection

HelixCode can automatically detect which format an LLM response uses.

### ### How It Works

The system checks for format-specific markers:

```
| Format | Detection Markers | |
|---|---|---|
| Whole-File | ` `` ` + `File:` |
| Diff | `---`, `+++`, `@@` |
| UDiff | `diff --git`, `index` |
| Search/Replace | `SEARCH:`, `REPLACE:`, `<<<<<<` |
| Editor | `INSERT AT LINE`, `DELETE LINE`, `REPLACE LINE` |
| Architect | `CREATE FILE`, `MODIFY FILE`, `DELETE FILE` |
| Ask | `QUESTION:`, `PROPOSED CHANGE:`, `?` |
| Line Number | `|`, `:` with multiple lines |
```

### ### Confidence Scoring

Formats are detected based on marker presence:

- **\*\*High confidence\*\***: Multiple specific markers found
- **\*\*Medium confidence\*\***: Some markers found
- **\*\*Low confidence\*\***: Generic markers only

### ### Manual Override

```

```go
// Force specific format
edits, err := registry.ParseWithFormat(ctx, formats.FormatTypeWhole, content)

// Or let auto-detection work
edits, format, err := registry.ParseWithAutoDetect(ctx, content)

```

/

e

/

e

Detection Priority

When multiple formats match: 1. UDiff (most specific markers) 2. Diff 3. Search/Replace 4. Editor 5. Line Number 6. Architect 7. Ask 8. Whole-File (fallback)

Best Practices

1. Choose the Right Format

For small changes: Search/Replace or Editor

```
File: config.yaml
search: timeout: 30
replace: timeout: 60
```

For new files: Whole-File or Architect

```
CREATE FILE: src/models/user.go
```go
package models
...
```

**\*\*For refactoring\*\*:** Architect or Ask

PROPOSED CHANGE: File: src/auth/ Description: Refactor authentication to use OAuth2 Rationale: Improve security and enable SSO

### ### 2. Be Explicit

**\*\*Good:\*\***

```
File: src/main.go <<<<<< SEARCH func calculateTotal(items []Item) float64
{ total := 0.0 for , item := range items { total += item.Price } return total } =====
func calculateTotal(items []Item, tax float64) float64 { total := 0.0 for , item := range
items { total += item.Price * (1 + tax) } return total } >>>>>> REPLACE
```

**\*\*Avoid:\*\***

Update calculateTotal to include tax

### ### 3. Provide Context

**\*\*Diff format - show context:\*\***

```
@@ -10,7 +10,8 @@ func calculateTotal(items []Item) float64 { total := 0.0 for _,
item := range items { - total += item.Price + // Apply sales tax + total += item.Price *
1.08 } return total }
```

### ### 4. Use Ask Mode When Unsure

QUESTION: Should I use pointer receivers or value receivers for this struct? File: src/models/user.go Context: User struct with many fields, will be copied frequently

PROPOSED CHANGE: File: src/models/user.go Description: Change all methods to use pointer receivers Rationale: Avoid copying large struct, more efficient

### ### 5. Group Related Changes

**\*\*Architect format for multiple files:\*\***

CREATE FILE: src/middleware/auth.go

...

MODIFY FILE: src/main.go Changes: - Add auth middleware import - Register middleware

DELETE FILE: src/old\_auth.go

### ### 6. Verify Search Patterns

Search/Replace requires **\*\*exact\*\*** matches:

**\*\*Wrong:\*\***

SEARCH: func calculateTotal(items []Item) float64 {  
(extra space)

**\*\*Correct:\*\***

SEARCH: func calculateTotal(items []Item) float64 {

### ### 7. Use Line Numbers Carefully

Always verify current line numbers:

```
```bash
# Get current line numbers
cat -n src/main.go

# Then use editor format
File: src/main.go
INSERT AT LINE 5:
import "os"
```

Troubleshooting

Format Not Detected

Problem: Auto-detection fails

Solution: 1. Add explicit markers: `File: test.gogo ...`

1. Or specify format manually:

```
edits, err := registry.ParseWithFormat(ctx, FormatTypeWhole,
    content)
```

Search Pattern Not Found

Problem: search pattern not found in file

Solution: 1. Check exact whitespace: `bash cat -A file.go # Show all characters`

1. Copy exact text from file:

```
# Get exact content
sed -n '10,15p' file.go
```

2. Use diff/editor format instead

Line Numbers Wrong

Problem: Editor format edits wrong lines

Solution: 1. Get current line numbers: `bash cat -n file.go | grep "function name"`

1. Use diff format instead (more robust)

Diff Won't Apply

Problem: Context doesn't match

Solution: 1. Increase context lines 2. Use search/replace instead 3. Use whole-file format

Ambiguous Format

Problem: Multiple formats detected

Solution: 1. Add more specific markers 2. Remove conflicting syntax 3. Specify format explicitly

FAQ

Q: Which format should I use? A: Start with search/replace for simple changes, diff for precise edits, and whole-file for small files.

Q: Can I mix formats in one response? A: No, each LLM response should use one format. Multiple files can be edited, but use same format.

Q: How does auto-detection work? A: It looks for format-specific markers (like @@ for diff, SEARCH: for search/replace) and chooses best match.

Q: What if auto-detection picks wrong format? A: Specify format explicitly in your prompt: "Use diff format to update..."

Q: Do all LLM providers support all formats? A: Yes, formats are independent of providers. All 13 providers can use any format.

Q: Can I create custom formats? A: Yes, implement the EditFormat interface and register with `registry.Register(yourFormat)`.

Q: Which format is most token-efficient? A: Search/Replace for small changes, Editor for line-specific edits. Whole-file uses most tokens.

Q: Can ask mode be automated? A: No, ask mode requires user interaction. Use other formats for automation.

Q: How do I handle merge conflicts? A: Use ask mode to get clarification, or architect mode to describe intended resolution.

Q: What's the difference between diff and udiff? A: UDiff includes git metadata (index, mode) and supports file operations (create/delete/rename). Diff is simpler.

Q: Can I use regular expressions in search/replace? A: The pattern matching is literal by default. Regex support depends on implementation configuration.

Q: How precise are line numbers in editor format? A: Very precise, but they must match current file state. Numbers shift when lines are added/removed.

See Also

- [@ Mentions User Guide](#)
 - [Slash Commands User Guide](#)
 - [Model Aliases User Guide](#)
 - [HelixCode API Documentation](#)
-

Document Version: 1.0 **Last Updated:** November 7, 2025 **Feedback:** [GitHub Issues](#)